

Integrating New Modelling Approaches

Your Name

2025-06-03

This document outlines key considerations and a recommended structure for incorporating new modeling approaches into the **REndo** R package. The goal is to maintain consistency, robustness, and ease of use, drawing lessons from the existing framework, including the recent **tscope** implementation.

1 Core Function Implementation

The heart of any new method is its main user-facing function (e.g., **tscope()**).

- **File Structure:** Place the main function in its own R file, typically named after the method (e.g., `R/f_yourmethod.R`).
- **Function Signature:**
 - Standard arguments should include `formula` and `data`.
 - Include a `verbose = TRUE` argument to allow users to see intermediate messages or suppress them.
 - Use `...` for extensibility if underlying functions (like `optimx` in `copulaCorrection` or `lm`) accept further arguments.
- **Initial Steps:**
 - Capture the function call: `cl <- match.call()`. This is crucial for reproducibility and display.
 - Perform input validation early (see Section 3).
- **Formula Processing:**
 - Utilize the `Formula` package (`F.formula <- Formula::as.Formula(formula)`) to handle potentially complex model formulas, especially those with multiple parts (e.g., `y ~ x1 + p1 | p2` for **tscope**). *(Self-correction: The original PDF text `y x1 + p1 | p1` was likely a typo and `y ~ x1 + p1 | p2` or similar is more standard for two-part formulas where `p1` and `p2` are different sets of variables)*
 - Extract the model frame: `mf <- stats::model.frame(formula = F.formula, data = data)`.
 - Obtain the response variable: `vec.data.y <- stats::model.response(mf)`.
 - Construct model matrices for different parts of the formula as needed: `X <- stats::model.matrix(F.formula, data = data, rhs = 1)`.
- **Core Estimation Logic:**
 - Clearly implement the statistical methodology.
 - If the method involves multiple stages (like **tscope**'s transformation/residualization and final regression), structure the code to clearly delineate these stages.
 - Store intermediate results that might be useful for diagnostics or detailed output in a `details` list.
- **Return Value:** The main function should conclude by calling the dedicated constructor function (see Section 2) to assemble and return the final object.

2 Constructor Function for Model Objects

Each new modeling approach should have a dedicated constructor function to create the specific result object.

- **File Structure:** Place the constructor in its own file (e.g., `R/f_new_rendo_yourmethod.R`).
- **Naming Convention:** `new_rendo_yourmethod()` (e.g., `new_rendo_tscope`, `new_rendo_copulacorrection`).

- **Functionality:**
 - This function takes all the computed pieces (coefficients, residuals, fitted values, model details, etc.) as arguments.
 - It primarily calls the base constructor `rendo:::new_rendo_base` (from `R/f_new_rendobase.R`) or assembles a list with a defined structure.
 - It assigns a class vector to the object, typically `c("rendo.yourmethod", "rendo.base")`. This enables method-specific S3 dispatch while inheriting common behaviors from `rendo.base`.
- **Standard Object Components** (to be passed to `new_rendo_base` or included in the list):
 - `call`: The original function call.
 - `F.formula`: The processed `Formula` object.
 - `mf`: The model frame.
 - `coefficients`: A named vector of all estimated parameters, including main effects and any auxiliary parameters (e.g., `rho...`, `sdError` for `tscope`).
 - `names.main.coefs`: A character vector of the names of the primary regression coefficients (excluding auxiliary ones).
 - `fitted.values`: Fitted values from the final model stage.
 - `residuals`: Residuals from the final model stage.
 - `(yourmethod).model` (e.g., `tscope.model`): The core model object from the final estimation stage if applicable (e.g., an `lm` object).
 - `details`: A list containing any other relevant intermediate calculations, transformed variables, or diagnostic information.

3 Robust Input Validation

Thorough input validation is critical for user experience and stability.

- **File Structure:** Group input checking functions in a dedicated file (e.g., `R/f_checkinput_yourmethod.R`, like `R/f_checkinput_tscope.R`).
- **Modular Checks:** Create small, focused functions for checking individual parameters or specific conditions.
 - Example: `checkinput_tscope_formula()`, `checkinput_tscope_data()`, `checkinput_tscope_verbose()`.
- **Helper Functions:** Utilize general-purpose input checking helpers (e.g., from `R/f_checkinput_helperfunctions.R`) for common tasks like:
 - Checking if `data` is a data frame.
 - Verifying a parameter is a single logical value.
 - Ensuring all formula variables are present in the data and are numeric where required.
- **Error Handling:**
 - Input validation functions should return a character vector of error messages. An empty vector indicates success.
 - The main function should call these validation functions and stop execution if errors are found, typically using a helper like `check_err_msg()` which might be internal to `REndo`.
- **Specific Checks:**
 - **Formula:** Correct structure (e.g., number of parts), presence of response and predictors. For `tscope`, it must be a two-part formula with the second part specifying endogenous regressors.
 - **Data:** Non-empty `data.frame` or `tibble`.
 - **Data vs. Formula:** All variables in the formula must exist in `data`. Columns used in calculations must be of appropriate types (e.g., numeric for regressors).
 - **Method-specific parameters:** Validate types, ranges, and interdependencies of any additional parameters.

4 S3 Methods for Consistency

Implement S3 methods for standard generic functions to ensure consistent behavior across `REndo` objects.

- **Inheritance:** Leverage `rendo.base` for common S3 methods. The file `R/f_s3_rendobase.R` provides base implementations for:
 - `coef.rendo.base`
 - `fitted.rendo.base`
 - `residuals.rendo.base`
 - `nobs.rendo.base`
 - `print.rendo.base`
 - `confint.rendo.base` (provides confidence intervals based on normality, ensure this is appropriate or provide a warning/alternative).
 - `case.names.rendo.base`
- **Method-Specific S3 Functions:**
 - If the default behavior from `rendo.base` is insufficient or misleading, implement a method-specific version (e.g., `summary.rendo.yourmethod`, `vcov.rendo.yourmethod`).
 - `summary()`: This should provide a more detailed output than `print()`. For methods with non-standard inference (like `tscope` where bootstrapping is recommended for SEs), the summary should clearly state any caveats regarding the displayed standard errors, t-values, and p-values.
 - `vcov()`: Provide a method to extract the variance-covariance matrix. Again, ensure its statistical validity for the given method.
 - `predict()`: Implement if prediction is meaningful for the method. Note the `tscope` documentation indicates no `predict` method is currently required.
- **File Structure:** Method-specific S3 functions can be grouped in a file like `R/f_s3_rendoyourmethod.R` (e.g., `R/f_s3_rendocopulacorrection.R`).

5 Comprehensive Documentation (Roxygen2)

Clear and thorough documentation is essential. Use Roxygen2 for generating `.Rd` files.

- **Main Function (`yourmethod.Rd`):**
 - `@title`, `@aliases`
 - `@description`: What the method does and its purpose.
 - `@usage`: Function signature.
 - `@param`: Detailed description of each argument (type, purpose, default values). Consider using roxygen templates for common sets of parameters like `formula`, `data`, `verbose`.
 - `@details`: In-depth explanation of the methodology, statistical background, algorithms, and any important assumptions or limitations. Use LaTeX for mathematical equations (e.g., `\deqn{}` or `\ifelse{html}{...}{...}` for web/pdf rendering, as seen in `tscope.Rd`. In LaTeX, this would typically be `$$...$$`).
 - `@return`: Describe the structure of the returned object (the one created by the constructor) and its components in detail.
 - `@references`: Cite relevant academic papers.
 - `@seealso`: Link to other relevant functions within REndo or other packages.
 - `@importFrom package function`: Ensure all external functions used are properly imported.
 - `@export`: Export the user-facing function.
 - `@examples`: Provide clear, runnable examples covering typical use cases and variations.
- **Constructor Function (`new_rendo_yourmethod.Rd`):** Document its arguments and the object it constructs, similar to `new_rendo_tscope.Rd`.
- **Datasets (`dataYourMethod.Rd`):** If example datasets are included, document them thoroughly (source, variables, structure) using `R/data.R` and generating corresponding `.Rd` files (e.g., `dataTscope.Rd`).

6 Testing (testthat)

Unit tests are crucial for verifying correctness and preventing regressions.

- **Framework:** Use the `testthat` package.

- **File Structure:** Create test files in `tests/testthat/` directory, typically named `test-inputchecks_yourmethod.R` and `test-correctness_yourmethod.R`.
- **Input Check Tests (`test-inputchecks_yourmethod.R`):**
 - Verify that valid inputs pass without error.
 - Ensure invalid inputs (wrong types, incorrect formula structures, missing data, etc.) trigger the expected error messages from your validation functions.
 - Example: `tests/testthat/test-inputchecks_tscope.R`.
- **Correctness Tests (`test-correctness_yourmethod.R`):**
 - **Known Values:** Compare results against known, pre-calculated values (e.g., from textbooks, other software, or published examples), as seen in `tests/testthat/test-correctness_tscope.R`.
 - **Invariance:** Test that results are robust to data permutations (e.g., row order shuffling).
 - **Edge Cases:** Test scenarios like no intercept, single predictor, collinearity (if applicable), small sample sizes.
 - **Formula Variations:** Test different valid formula specifications (transformations, interactions).
 - **Internal Consistency:** Check that dimensions of returned matrices/vectors are correct, and that components of the returned object are as expected.
 - **S3 Method Tests:** If custom S3 methods are implemented, test their output and behavior directly. Helper files like `helper_test_s3methods_rendobase.R` can provide common testing patterns.
- **Runnability Tests:** For more complex models, simple tests that ensure the model runs to completion with various valid configurations without error can be valuable (e.g., `tests/testthat/test-runability_copulacorrect`).

7 Namespace Management

- Use `@importFrom package` function in roxygen comments to manage dependencies. This helps roxygen2 generate the `NAMESPACE` file correctly.
- Explicitly `@export` user-facing functions and S3 methods.

8 Rcpp Integration (If Applicable)

If computationally intensive parts are written in C++ using Rcpp (as seen in `copulaCorrection`):

- Ensure Rcpp code is well-documented and tested.
- Manage exports from C++ to R carefully via `RcppExports.cpp` and `RcppExports.R`.

9 Vignettes and Overall Package Documentation

- Consider adding or updating vignettes (`vignettes/`) to demonstrate the use of the new method in a broader context, perhaps comparing it with other approaches.
- Ensure the main package documentation (`REndo-package.R`) and `README.md` are updated to reflect the new functionality.

By adhering to these guidelines, new contributions to `REndo` will align with the existing package structure, ensuring quality, maintainability, and a consistent user experience. The `tscope` implementation serves as a strong template for many of these points.